

ToSh Cheat Sheet: Collections And Pipelines

Literals, typed containers, projection, grouping, and summaries

spec + examples + tests

Collection Literals

# arrays [1, 2, 3] ["alpha", "beta"]	
# record-like dynamic objects { Name = "Komrad", Uid = 1000 }	
# sets {: "alpha", "beta", "alpha" :}	
# tuples (42, "hello")	

Literal	Runtime shape
[...]	array
{ Name = Value }	dynamic record
{: ... :}	set with de-duplication
(..., ...)	tuple with Item1, Item2, ...

Constructed Collections

new list<int>(10, 20, 30)	
new array<int>(1, 2, 3)	
new dict<string, int>(Alice, 95, Bob, 87)	
new map({ Theme = "dark", TabSize = 4 })	
new hashtable(HOME, "/home/me", SHELL, "/usr/bin/tosh")	
new table(Name, "Toast", Uid, 1000)	
new tuple(alpha, 42)	

- list<T> is growable and works well for accumulation.
- dict<K, V> gives a strongly typed dictionary.
- map(...) and record literals are handy for dynamic data.
- table is useful when you want a record that renders well.

Basic Access

var words = ["hello", "world"] echo \$words[0] echo \$words.Length	
var pair = (42, "hello") echo \$pair.Item2	
var person = { Name = "Komrad", Uid = 1000 } echo \$person.Name get-props \$person has-prop \$person Name	

Pipeline Core

Command	Use
where	filter rows with a predicate
get / pick / select	project members or indices
each	run a block per item
map	transform each item
flat-map	transform and flatten
flatten	expand one nesting level
sort, reverse	reorder results
first, last, skip	slice a stream

ls -la where _.Type == file get { Name, Size } echo "Hello" "World" each { _.ToLower() } echo [1, 2, 3] flatten map { _ * 2 }	
---	--

Inside pipeline predicates, _ is the current item. In many places a block like { _.Size > 1kb } is easier to read than a named lambda.	
--	--

Grouping And Stats

ls -la group-by Type ls -la frequencies ls -la distinct	
seq 5 summarize --sum --avg --min --max --count df summarize _.Used ps summarize --avg WorkingSet --max WorkingSet	

Command	Best for
group-by	collect rows by key
group-while	consecutive runs
frequencies	histogram-like counts
summary / summarize	aggregates over scalars or columns

Structured Text

echo "[{\"name\":\"alpha\",\"score\":95}]" from json flatten echo "a,b\\n1,2" from csv echo "toast" hash sha256 echo "the quick brown fox" grep "quick"	
\$items to json \$rows to csv \$rows template "{{ Name }}" => "{{ Quantity }}"	

Collection Patterns

var items = new list<string>("bread", "coffee", "soap") \$items where _.Length > 4 sort each { echo _ }	
var report = new list() ls -la where _.Type == file each { \$report.Add({ Name = _.Name, Size = _.Size }) ignore } \$report sort Size reverse	

Transform Recipes

# top 5 files by size ls -la where _.Type == file sort Size reverse first 5	
# names only ls -la get Name	
# files grouped by extension ls -la where _.Type == file group-by Extension	
# flatten JSON rows then rank them echo "[{\"name\":\"a\",\"score\":1},{\"name\":\"b\",\"score\":2}]" from json flatten sort score reverse get { name, score }	

Small But Handy

Tool	Why it matters
collect	gather a pipeline into one list
partition	split into matching/non-matching lists
rename	rename projected fields
transpose	pivot rows into columns
zip, window, chunk	sequence reshaping

ToSh sweet spot: load typed rows, project only the fields you care about, then summarize or regroup them without dropping down to fragile text parsing.

ToSh Cheat Sheet: Collections And Pipelines

Reshaping, folding, formatting, and advanced stream tools

Reshape Toolbox

Command	What it does
collect	buffer the whole pipeline into one array
chunk N	fixed-size batches
window N	sliding windows over neighbors
zip seq	pair items with another sequence
interleave seq	alternate values from two sequences
take-while / skip-while	prefix/suffix slicing by predicate

```
echo 1 2 3 | collect
echo 1 2 3 4 | chunk 2
echo 1 2 3 4 | window 3
echo 1 2 3 | zip [10, 20, 30]
```

Fold And Trace

```
echo 1 2 3 4 | reduce 0 func(acc, x) => ($acc + $x)
echo one two three | reduce "" { $acc + _.Substring(0, 1) }
echo 1 2 3 4 | scan 0 { $acc + _ }
```

- **reduce** returns one final accumulator.
- **scan** yields every intermediate accumulator value.
- On empty input, **reduce** returns the seed unchanged.

Split And De-Dupe

```
var [evens, odds] = (echo 1 2 3 4 | partition { (_ % 2) == 0 })
echo alpha beta alpha gamma | distinct
echo alpha beta alpha gamma | frequencies
```

Stopping And Slicing

```
echo 1 2 3 4 | take-while { _ < 3 }
echo 1 2 3 4 | skip-while { _ < 3 }
echo alpha beta gamma | first 2
echo alpha beta gamma | last 2
```

Projection Cleanup

```
ls -la |
get { Name, Size, Modified } |
rename Name FileName Size Bytes

echo { A = 1, B = 2 } { A = 3, B = 4 } | transpose
```

Command	Best for
rename	clean projected field names
transpose	rows-to-columns pivoting
pick/select	aliases for get
tee	pass through while also capturing or writing

Structured Format Bridges

```
echo "{\"name\":\"toast\"}" | from json
curl https://example/api | from json | flatten
cat data.csv | from csv
```

```
ls | to json
ls | to csv
ls | to json --compact
```

from and **to** keep data as CLR objects until you explicitly **flatten** or **stringify** it, which is why downstream **get**, **where**, and **summarize** still work.

Text Templates

```
$rows | template "{{ Name }}" => {{ Quantity }}"
$rows | template "{{ Category }}" / {{ Name }}"
```

Capture And Replay

```
ls | tee -v snapshot | first 3
$snapshot | get Name
ls | to json | tee report.json | count
```

Pipeline Recipes

```
func bigFiles(minSize: StorageSize) {
    ls -la | where _.Type == file |
        where _.Size >= $minSize |
            sort Size | reverse
}
```

```
echo "[{\"name\":\"alpha\",\"score\":95},{\"name\":\"beta\",\"score\":87}]" |
from json | flatten | sort score | reverse | get { name, score }
```

Grouping Recipes

```
ls | group-by Extension
ps | group-by func(p) => ($p.Name.Substring(0, 1))
echo 1 2 3 10 11 | group-while { _ < 10 }
```

group-by groups by key across the whole stream, while **group-while** is about consecutive runs. Use the latter when input order already carries meaning.

Summary Patterns

```
df | summarize
df | summarize _.Used
seq 5 | summarize --sum --avg --min --max --count
ps | summarize --avg Memory --max Memory
```

- **sum**, **avg**, **min**, and **max** are great for one target.
- **summarize** shines when you want multiple aggregates or structured column summaries back.

Generated Sequences

```
iterate 1 { _ * 2 } | take 6
unfold 5 { if _ > 0 { [_, _ - 1] } else { null } }
converge " toast " { _.Trim() }
```

iterate grows a stream from a seed, **unfold** emits value/state pairs until null, and **converge** keeps normalizing until two consecutive results are equal.